

Reserving PyPI Names: **drydock** and **marina**

How to claim the package names **drydock** and **marina** on the Python Package Index (PyPI).

Status (checked 2026-06-10)

Name	PyPI (production)	TestPyPI
drydock	TAKEN — abandoned "Docker cluster construction utility", last release 2014 (v0.6.10), owner <i>Nekroze</i>	AVAILABLE
marina	TAKEN — abandoned "marina manages docker instances", last release 2020 (v0.4.3), owner <i>mmerickel</i>	AVAILABLE

Neither bare name is free on production PyPI. Both holders are inactive Docker-era utilities, which makes them candidates for a PEP 541 transfer — but that is a request, not a guarantee. The reliable move is to claim a fallback distribution name you control immediately, and optionally pursue the bare name through PEP 541 in parallel.

0. Concepts you need first

- **Distribution name vs import name.** The *distribution name* is what you `pip install`; it must be globally unique on PyPI (e.g. **drydock**). The *import name* is the Python package you `import` (e.g. `import drydock`). They do **not** have to match. A project published as **drydock-sdd** can still install an importable **drydock** package and a **drydock** console command. A fallback PyPI name therefore costs nothing at the code or CLI level.
- **There is no "reservation" feature.** You claim a name by being the first to upload a release for it. PEP 541 governs disputes over names that already exist.
- **PyPI and TestPyPI are independent.** A name free on one may be taken on the other. Claim both where you can. TestPyPI is also the safe place to rehearse the upload.
- **Anti-squatting policy.** PyPI prohibits reserving names you do not intend to use. A minimal but genuine placeholder for a project under active development is acceptable; an empty land-grab is not.

1. Decide your naming strategy

Both bare names are taken, so choose one path per project. The import name and CLI stay **drydock** / **marina** regardless — only the PyPI distribution name changes.

Option	drydock	marina
A. Fallback name now (recommended)	drydock-sdd , drydock-cli , or webcloud-drydock	marina-sdd , marina-lib , or webcloud-marina
B. PEP 541 transfer request	strong case (dead since 2014)	plausible (dead since 2020)
C. Both	claim a fallback now; file PEP 541 in parallel	same

2. One-time account setup

1. Create accounts: <https://pypi.org/account/register/> and <https://test.pypi.org/account/register/>.
2. Enable two-factor authentication on both (required to upload).
3. Create a scoped API token on each (Account settings → API tokens). Scope "Entire account" for the first upload; re-scope to the project once it exists.

Store tokens in `~/.pypirc` (never commit this file):

```
[distutils]
index-servers = pypi testpypi

[pypi]
username = __token__
password = pypi-AgEI...           # your PyPI token

[testpypi]
repository = https://test.pypi.org/legacy/
username = __token__
password = pypi-AgEI...           # your TestPyPI token
```

3. Build a minimal placeholder package

Do this in a **throwaway directory**, not in the Drydock repository. For a proprietary project you do not want to publish real source merely to hold a name, so the placeholder ships an empty package.

```
mkdir -p ~/tmp/reserve-marina/src/marina && cd ~/tmp/reserve-marina
printf '""Reserved for Web Cloud Studio Marina.""\n__version__ = "0.0.0"\n' >
src/marina/__init__.py
printf '# Marina\n\nReserved for Web Cloud Studio. Project under active
development.\n' > README.md
```

`pyproject.toml`:

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "marina"                # or your fallback, e.g. "marina-sdd"
version = "0.0.0"
description = "Reserved for Web Cloud Studio. Project under active development."
readme = "README.md"
requires-python = ">=3.11"
authors = [{ name = "Ed Barlow", email = "edward.m.barlow@gmail.com" }]
license = { text = "Proprietary" }
```

```
[tool.hatch.build.targets.wheel]
packages = ["src/marina"]
```

Then build:

```
uv build                                # produces dist/*.whl and dist/*.tar.gz
```

Repeat in a separate throwaway directory for **drydock** (or your drydock fallback). Do **not** reuse the real Drydock source for a placeholder.

4. Claim TestPyPI first (free for both; rehearses the flow)

```
uv publish --publish-url https://test.pypi.org/legacy/ --token <TESTPYPI_TOKEN>
dist/*
# equivalent: twine upload -r testpypi dist/*
```

Verify: <https://test.pypi.org/project/marina/>

5. Claim the production name (fallback)

The fallback name is free on production PyPI, so upload there:

```
uv publish --token <PYPI_TOKEN> dist/*
# equivalent: twine upload dist/*
```

Verify the project page, then re-scope the API token to that project only.

6. (Optional) PEP 541 request for the bare names

To pursue the exact names **drydock** / **marina**:

1. Confirm inactivity (done: drydock 2014, marina 2020).
2. Attempt to contact the current owner (email or a GitHub issue on their repo) and **document** it.
3. File a request at <https://github.com/pypi/support/issues> using the project-name-request / PEP 541 template. Include: the name, evidence of abandonment, your contact attempts, and your intended active use.
4. PyPI admins evaluate per PEP 541. Outcome and timing are not guaranteed.

Reference: <https://peps.python.org/pep-0541/>

7. After you hold a name

- Switch the real project to **Trusted Publishing (OIDC)** from GitHub Actions — no stored tokens. Configure at PyPI → project → Publishing, matching repository **webcloudstudio/Drydock**, the release

workflow, and a `pypi` environment.

- When the real release is ready, upload it over the `0.0.0` placeholder (a higher version supersedes it); optionally `yank` the placeholder.
- Repeat the Trusted Publishing setup for Marina.

Appendix — re-check availability any time

```
for n in drydock marina; do
  for base in https://pypi.org/pypi https://test.pypi.org/pypi; do
    code=$(curl -s -o /dev/null -w "%{http_code}" "$base/$n/json")
    echo "$base/$n -> $code    (200 = taken, 404 = available)"
  done
done
```